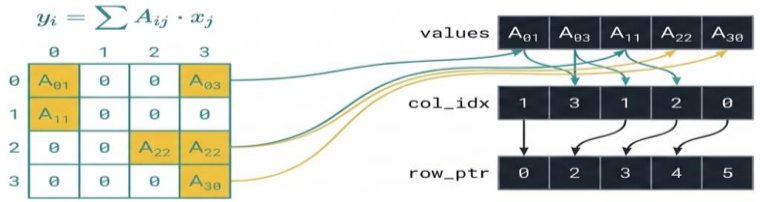
Concept Overview: Sparse Matrix–Vector Multiplication

What SpMV computes

$$y = A\,x$$

- ***A*** is a sparse $m \times n$ matrix. Most entries are zero.
- ***x*** and ***y*** are dense vectors.
- Each nonzero costs 1 multiply and 1 add: about **2 · nnz** FLOPs per product.
- Storage formats: COO, CSR, DIA, ELL. The best choice depends on matrix structure and hardware [2].
- This study uses **CSR**.

CSR format: 4 × 4 example



A

1	0	**6**	0
0	**2**	0	**7**
0	0	**3**	0
5	0	0	**4**

row_ptr	**0**	**2**	**4**	**5**	**7**		
col_idx	**0**	**2**	**1**	**3**	**2**	**0**	**3**
values	**1**	**6**	**2**	**7**	**3**	**5**	**4**

Row *i* owns entries `row_ptr[i]` to `row_ptr[i+1]−1`. Only nonzeros are stored.

Why it is hard to speed up

Low arithmetic intensity

$2\ \text{FLOP} / 12\ \text{B} \approx 0.17\ \text{FLOP/B}$

Per nonzero: 8-byte value + 4-byte index. Memory traffic sets the pace [1], [3].

Irregular access to x

`col_idx` decides which entries of *x* each row reads, so accesses scatter across memory [1], [2].

Uneven row lengths

Equal row counts can carry unequal nonzero work across threads [1].

Row kernel

```
for i = 0 … m−1:
    y[i] = Σ values[j] · x[col_idx[j]],
    j = row_ptr[i] … row_ptr[i+1]−1
```